

Reviewer Pack — Minimal Order of Eta-Invariant and Resolution Proof Width In...

Entry 39831ff7bba7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 01:00:31 UTC

Minimal Order of Eta-Invariant and Resolution Proof Width Inequality

Entry ID: 39831ff7bba7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 01:00:31 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Local Indeterminacy Theory) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF φ with n variables, the minimal order of the eta-invariant ($\eta(\varphi)$) for its associated variety is linearly correlated with its resolution proof width $w(\varphi)$, such that $\eta(\varphi) = O(w(\varphi))$. Furthermore, there exists a constant $c > 0$ such that for all instances φ , $\eta(\varphi) \geq cw(\varphi)$.

Rationale (proposer's reasoning):

The eta-invariant captures the local indeterminacy of algebraic varieties and could potentially provide a measure of the complexity of the resolution process. A strong correlation between this invariant and resolution proof width could offer new insights into the structure of proof systems and their ability to handle different types of formulas.

Taxonomy category: INCONCLUSIVE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9a69cd25eb6b5d76

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each CNF φ , if the ratio of $\eta(\varphi)$ to $w(\varphi)$ is within the bounds $O(w(\varphi))$ and $cw(\varphi)$, where $c > 0$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 6 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal order of eta-invariant" AND "resolution proof width" AND algebraic geometry" - "eta-invariant" [term] resolution proof complexity" - algebraic geometry local indeterminacy theory AND resolution proof complexity AND eta-invariant

Top relevant hits considered: - [http://arxiv.org/abs/nlin/0101006v1] On Complexity and Emergence - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/1607.00443v1] Algebraic Proof Complexity: Progress, Frontiers and Challenges - [http://arxiv.org/abs/2009.02774v5] Representation theory of finite groups through (basic) algebraic geometry - [http://arxiv.org/abs/math/0502387v1] Multiplier ideals in algebraic geometry - [http://arxiv.org/abs/1912.00347v3] Equiresidual algebraic geometry I: The affine theory

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        n = len(matrix)
        for i in range(n):
            max_row = i
            for j in range(i + 1, n):
                if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                    max_row = j
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            factor = matrix[i][i]
            for j in range(n):
                matrix[i][j] /= factor
            for j in range(n):
                if j != i:
                    factor = matrix[j][i]
                    for k in range(n):
                        matrix[j][k] -= factor * matrix[i][k]
        return matrix

    def resolution_width(cnf):
        clauses = cnf.split('\n')
        n = len(clauses)
```

```

visited = set()
stack = []
for clause in clauses:
    literals = [int(lit) for lit in clause.split() if lit != '0']
    for literal in literals:
        if -literal in visited:
            return 1
        visited.add(literal)
        stack.append((literals, visited.copy()))
return len(stack)

def eta_invariant(cnf):
    clauses = cnf.split('\n')
    n = len(clauses)
    matrix = [[0] * (n + 1) for _ in range(n + 1)]
    for i, clause in enumerate(clauses):
        literals = [int(lit) for lit in clause.split() if lit != '0']
        for literal in literals:
            matrix[i][abs(literal)] += 1
    gaussian_elimination(matrix)
    rank = sum(1 for row in matrix if any(row[j] != 0 for j in range(n + 1)))
    return n - rank

instances_tested = 0
eta_values = []
width_values = []
n_max = 5

for _ in range(30):
    n = random.randint(5, 40)
    cnf = '\n'.join(' '.join(str(random.choice([-1, 1]) * i) for i in range(1, n + 1)) for _ in range(n))
    eta_phi = eta_invariant(cnf)
    w_phi = resolution_width(cnf)

    if eta_phi is not None and w_phi is not None:
        instances_tested += 1
        eta_values.append(eta_phi)
        width_values.append(w_phi)
        n_max = max(n_max, n)

if instances_tested < 30:
    return {
        "metric_name": "eta_invariant_resolution_width_ratio",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

mean_eta = sum(eta_values) / instances_tested
std_eta = math.sqrt(sum((x - mean_eta) ** 2 for x in eta_values) / instances_tested)
mean_width = sum(width_values) / instances_tested
std_width = math.sqrt(sum((x - mean_width) ** 2 for x in width_values) / instances_tested)

ratio_mean = mean_eta / mean_width

```

```

ratio_std = std_eta / mean_width

return {
    "metric_name": "eta_invariant_resolution_width_ratio",
    "metric_value": ratio_mean,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": ratio_mean >= 1 and ratio_std <= 0.5,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_ratio = sum(result["metric_value"] for result in results if result["metric_value"] is not None)
    std_ratio = math.sqrt(sum((result["metric_value"] - mean_ratio) ** 2 for result in results if result["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["metric_value"] is not None for result in results):
        print(f"RESULT: SUPPORTED mean={mean_ratio} std={std_ratio} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient={mean_ratio}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ldfefdda.py", line 114, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ldfefdda.py", line 73, in run_trial
    eta_phi = eta_invariant(cnf)
               ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ldfefdda.py", line 61, in eta_invariant
    gaussian_elimination(matrix)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ldfefdda.py", line 31, in gaussian_elimination
    matrix[i][j] /= factor
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution due to a division by zero error, which prevents us from verifying the conjecture’s conditions. | next: Investigate and fix the division by zero issue in the test code to allow for proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	47361
2	preregistration	ollama_remote	glm4:latest	0	0	9670
3	novelty	ollama_remote	glm4:latest	0	0	9022
4	novelty	ollama_remote	glm4:latest	0	0	14227
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19739
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10890
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	64124
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24052
9	judge	ollama_remote	glm4:latest	0	0	98487

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 297572 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/39831ff7bba7.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/39831ff7bba7.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/39831ff7bba7.tar.gz* (if generated)